

Compendium: Matrix Decompositions in Practice

Full numerical analysis of LU and Cholesky methods with examples

1. Justification: Why don't we use A^{-1} ?

In engineering (especially in the Finite Element Method), we look for the displacement vector $\mathbf{x}$ by solving the giant system $\mathbf{Ax} = \mathbf{b}$. A naive approach is $\mathbf{x} = \mathbf{A}^{-1}\mathbf{b}$.

Why is this numerical suicide?

1. Inverting the matrix $\mathbf{A}^{-1}$ costs the processor $\mathcal{O}(N^3)$ operations.
2. Engineering matrices are 99% full of zeros (sparse matrices). Inverting such a matrix generates a dense matrix. Your computer will immediately throw an out-of-memory error (*MemoryError*).

Solution: Factorization (decomposition). Instead of inverting $\mathbf{A}$, we break it down into a product of triangular matrices. We then solve the system of equations at a low cost of $\mathcal{O}(N^2)$ using forward and backward substitution.

2. LU Decomposition (For general square matrices)

We decompose $\mathbf{A} = \mathbf{LU}$, where $\mathbf{L}$ is a lower triangular matrix (with ones on the diagonal), and $\mathbf{U}$ is an upper triangular matrix. Computational cost: $\sim \frac{2}{3}N^3$ FLOPS (floating-point operations).

Analytical Example (Step by Step)

Let's decompose the matrix $A = \begin{bmatrix} 2 & 3 \\ 4 & 7 \end{bmatrix}$.

We are looking for elements such that:

$$\begin{bmatrix} 2 & 3 \\ 4 & 7 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ l_{21} & 1 \end{bmatrix} \begin{bmatrix} u_{11} & u_{12} \\ 0 & u_{22} \end{bmatrix}$$

Multiplying the right side, we get a system of equations:

$$\begin{bmatrix} u_{11} & u_{12} \\ l_{21}u_{11} & l_{21}u_{12} + u_{22} \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 4 & 7 \end{bmatrix}$$

Solution:

1. From the first row, it is immediately obvious that $u_{11} = 2$ and $u_{12} = 3$.
2. From the bottom-left corner: $l_{21}u_{11} = 4 \implies l_{21} \cdot 2 = 4 \implies l_{21} = 2$.
3. From the bottom-right corner: $l_{21}u_{12} + u_{22} = 7 \implies 2 \cdot 3 + u_{22} = 7 \implies 6 + u_{22} = 7 \implies u_{22} = 1$.

Final result:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 2 & 3 \\ 0 & 1 \end{bmatrix}$$

3. Cholesky Decomposition (For SPD matrices)

If the matrix is Symmetric and Positive Definite ($\mathbf{A} = \mathbf{A}^T$ and the energy $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$), we can simplify the algorithm to $\mathbf{A} = \mathbf{L}\mathbf{L}^T$. Computational cost: $\sim \frac{1}{3}N^3$ FLOPS. Cholesky decomposition is **twice as fast** and uses half the memory of LU, because it only calculates one triangle.

Analytical Example (Step by Step)

Let's decompose the matrix $A = \begin{bmatrix} 4 & 12 \\ 12 & 45 \end{bmatrix}$.

We are looking for:

$$\begin{bmatrix} 4 & 12 \\ 12 & 45 \end{bmatrix} = \begin{bmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{bmatrix} \begin{bmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{bmatrix}$$

$$\begin{bmatrix} 4 & 12 \\ 12 & 45 \end{bmatrix} = \begin{bmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{21}l_{11} & l_{21}^2 + l_{22}^2 \end{bmatrix}$$

Solution:

1. From the top-left corner: $l_{11}^2 = 4 \implies l_{11} = 2$ (we always take the positive root).
2. From the bottom-left corner: $l_{21}l_{11} = 12 \implies l_{21} \cdot 2 = 12 \implies l_{21} = 6$.
3. From the bottom-right corner: $l_{21}^2 + l_{22}^2 = 45 \implies 36 + l_{22}^2 = 45 \implies l_{22}^2 = 9 \implies l_{22} = 3$.

Final result:

$$\mathbf{L} = \begin{bmatrix} 2 & 0 \\ 6 & 3 \end{bmatrix}$$

LU Decomposition (Doolittle's Algorithm) for dimensions $N \geq 3$

For square matrices of larger dimensions, solving the system by directly comparing terms becomes inefficient. In numerical analysis, the optimized **Doolittle's algorithm** is used, where we assume ones on the main diagonal of matrix $\mathbf{L}$ ($l_{ii} = 1$).

The general formulas for the elements of matrix **U** (upper) and **L** (lower) are as follows. For the upper triangle (elements u_{ij} for $i \leq j$):

$$u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj}$$

For the lower triangle (elements l_{ij} for $i > j$):

$$l_{ij} = \frac{1}{u_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} u_{kj} \right)$$

Step-by-step derivation: 3×3 Matrix

Given is a system with a specific matrix **A**:

$$\mathbf{A} = \begin{bmatrix} 2 & -1 & 1 \\ 4 & 1 & -1 \\ 2 & 2 & 1 \end{bmatrix}$$

Step 1: First row of U and first column of L The first row of **U** is simply the first row of **A**.

$$u_{11} = 2, \quad u_{12} = -1, \quad u_{13} = 1$$

The first column of **L** is the first column of **A** divided by u_{11} .

$$l_{21} = \frac{a_{21}}{u_{11}} = \frac{4}{2} = 2, \quad l_{31} = \frac{a_{31}}{u_{11}} = \frac{2}{2} = 1$$

Step 2: Second row of U and second column of L We calculate the row for **U**:

$$u_{22} = a_{22} - l_{21}u_{12} = 1 - 2(-1) = 3$$

$$u_{23} = a_{23} - l_{21}u_{13} = -1 - 2(1) = -3$$

We calculate the column for **L**:

$$l_{32} = \frac{a_{32} - l_{31}u_{12}}{u_{22}} = \frac{2 - 1(-1)}{3} = \frac{3}{3} = 1$$

Step 3: Third row of U We are left with the last element on the diagonal of **U**:

$$u_{33} = a_{33} - (l_{31}u_{13} + l_{32}u_{23}) = 1 - (1 \cdot 1 + 1 \cdot (-3)) = 1 - (1 - 3) = 3$$

Final result of the LU factorization:

$$\mathbf{L} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{U} = \begin{bmatrix} 2 & -1 & 1 \\ 0 & 3 & -3 \\ 0 & 0 & 3 \end{bmatrix}$$

Cholesky Decomposition for a 4×4 system

The complexity of the LU algorithm lies in the fact that we calculate $\mathbf{L}$ and $\mathbf{U}$ independently. If matrix $\mathbf{A}$ is Symmetric Positive Definite (SPD), then $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ holds.

The recursive formulas for the elements of the lower matrix $\mathbf{L}$ are fully self-contained: For the diagonal ($j = i$):

$$l_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} l_{jk}^2}$$

For elements below the diagonal ($i > j$):

$$l_{ij} = \frac{1}{l_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} l_{ik} l_{jk} \right)$$

Step-by-step derivation: 4×4 Matrix

Given is a three-dimensional stress tensor (in matrix form) with an added abstract fourth dimension, forming the matrix $\mathbf{A}$:

$$\mathbf{A} = \begin{bmatrix} 4 & 2 & -2 & 0 \\ 2 & 10 & 2 & 6 \\ -2 & 2 & 6 & 0 \\ 0 & 6 & 0 & 6 \end{bmatrix}$$

Column 1 ($j = 1$):

$$l_{11} = \sqrt{a_{11}} = \sqrt{4} = 2$$

$$l_{21} = \frac{a_{21}}{l_{11}} = \frac{2}{2} = 1$$

$$l_{31} = \frac{a_{31}}{l_{11}} = \frac{-2}{2} = -1$$

$$l_{41} = \frac{a_{41}}{l_{11}} = \frac{0}{2} = 0$$

Column 2 ($j = 2$):

$$l_{22} = \sqrt{a_{22} - l_{21}^2} = \sqrt{10 - 1^2} = \sqrt{9} = 3$$

$$l_{32} = \frac{a_{32} - l_{31}l_{21}}{l_{22}} = \frac{2 - (-1)(1)}{3} = \frac{3}{3} = 1$$

$$l_{42} = \frac{a_{42} - l_{41}l_{21}}{l_{22}} = \frac{6 - (0)(1)}{3} = \frac{6}{3} = 2$$

Column 3 ($j = 3$): Element on the diagonal:

$$l_{33} = \sqrt{a_{33} - (l_{31}^2 + l_{32}^2)} = \sqrt{6 - ((-1)^2 + 1^2)} = \sqrt{6 - 2} = \sqrt{4} = 2$$

Element below the diagonal:

$$l_{43} = \frac{a_{43} - (l_{41}l_{31} + l_{42}l_{32})}{l_{33}} = \frac{0 - (0(-1) + 2(1))}{2} = \frac{-2}{2} = -1$$

Column 4 ($j = 4$): Only the element on the diagonal, where we close the energy balance (all powers of elements from a given row from previous columns):

$$l_{44} = \sqrt{a_{44} - (l_{41}^2 + l_{42}^2 + l_{43}^2)} = \sqrt{6 - (0^2 + 2^2 + (-1)^2)} = \sqrt{6 - (0 + 4 + 1)} = \sqrt{1} = 1$$

Final result of the 4×4 Cholesky factorization:

$$\mathbf{L} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 1 & 3 & 0 & 0 \\ -1 & 1 & 2 & 0 \\ 0 & 2 & -1 & 1 \end{bmatrix}$$

This form of the algorithm (written in pseudocode as a triple loop with a sum accumulator) is exactly what has been implemented in C and Fortran libraries that form the foundation of `scipy.linalg.cholesky`.

4. Tasks for independent solving (Training)

Task 1 (LU Decomposition): Determine the matrices $\mathbf{L}$ and $\mathbf{U}$ for matrix $\mathbf{B}$:

$$\mathbf{B} = \begin{bmatrix} 3 & 1 \\ 6 & 5 \end{bmatrix}$$

Task 2 (Cholesky Decomposition): Determine the lower triangular matrix $\mathbf{L}$ for matrix $\mathbf{C}$:

$$\mathbf{C} = \begin{bmatrix} 9 & -3 \\ -3 & 5 \end{bmatrix}$$

Task 3 (Cholesky Trap): Try to perform the Cholesky decomposition for matrix $\mathbf{D}$. At which step will the algorithm "crash" numerically, and what does this mean in the context of this matrix's properties?

$$\mathbf{D} = \begin{bmatrix} 4 & 5 \\ 5 & 6 \end{bmatrix}$$

5. Python Test (SciPy)

The script below empirically proves that the complexity of Cholesky ($\frac{1}{3}N^3$) gives a two-fold speedup compared to LU ($\frac{2}{3}N^3$) for large systems.

```
1 import numpy as np
2 import scipy.linalg as la
3 import time
4
5 N = 2500
6
7 print("Matrix generation...")
8 A_random = np.random.rand(N, N)
9 # Multiplying a matrix by its transpose generates the SPD
property
10 A_spd = np.dot(A_random, A_random.T)
11
12 start_lu = time.perf_counter()
13 P, L, U = la.lu(A_spd)
14 time_lu = time.perf_counter() - start_lu
15 print(f"LU Time: {time_lu:.4f} sec")
16
17 start_chol = time.perf_counter()
18 L_chol = la.cholesky(A_spd, lower=True)
19 time_chol = time.perf_counter() - start_chol
20 print(f"Cholesky Time: {time_chol:.4f} sec")
21
22 print(f"Time ratio (LU/Cholesky): {time_lu / time_chol:.2f}x")
23
```

Answers to tasks from Section 4

Re 1: $\mathbf{L} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$, $\mathbf{U} = \begin{bmatrix} 3 & 1 \\ 0 & 3 \end{bmatrix}$

Re 2: $\mathbf{L} = \begin{bmatrix} 3 & 0 \\ -1 & 2 \end{bmatrix}$

Re 3: In the third step, we calculate $l_{22}^2 = a_{22} - l_{21}^2$. We have $l_{11} = 2$, hence $l_{21} = 2.5$. Substituting into the formula: $l_{22}^2 = 6 - (2.5)^2 = 6 - 6.25 = -0.25$. The computer will try to calculate the square root of a negative number and throw a `ValueError`. This is empirical proof that matrix **D** is **not** Positive Definite!